Principles of operating system: Acronym and Definition

Lecture 9, Memory management 1

Address Space(AS)

- Set of number that can be use as address in the main memory.
- Consist of Code(static text), Stack(static variable and function calls), Heap(dynamic memory allocation).

Virtural Memory(VM)

- More like private memory.
- Let each process has their own address space (virtual address), which will be map to physical address space using some protocal like base and bound.

Limited Direct Execution

- Let the program run directly on the hardware as much as possible with only little OS intervention, like system call, timer interupt or exception.
- 2 Goals. Effiency and control.

Address Translation

- Process of mapping virtual address to physical address.
- Some implementation require hardware support. Such as base and bound.
- Hardware support usually located at Memory Management Unit(MMU)

Base and bound(Dynamic Relocation)

- Require 2 registers as hardware support: `base` and `bound`.
- `base` stores starting location of the program in the physical memory.
- `bound` stores length of virtual memory. Used to check whether virtual address is invalid(exceed its size) or not. If it invalid, most of the time process is terminated.
- Both registers were kept in MMU.
- Require following support from OS: Memory management(Free list), base/bounds management upon context switching(Each process has their own `base` and `bound` value), Exception handling.

Segmentation

- Divied address into two parts. Segment and offset.
- Segment will identify whether address should be in code, stack or heap.
- Offset is offset position from the base address.
- Require following OS support: swap base address data and grows direction(it can grow negatively) for context swtich, update segment size.

Fragmentation Problem

- Fragmentation is when memory is free but too small for new processes, but if arrange correctly, new processes can use it.
- 2 types, External and Internal.

Heap and Memory Allocation

- `malloc()` is system call API for heap memory allocation.
- `free()` is system call API for free the allocated memory by putting them into free list.
- See `man malloc`(has info of both `malloc` and `free` in the same entry).

Free List

- Free list is a list that contain infomation heap memory that already `free()`'d.
- Each entry contain starting address and its length.
- Coalescing combines contiguous free'd memomy together. For example [(addr:10, len:10), (addr:0, len:10), (addr:20, len:10)] can be combine to [(addr:0, len:30)].

Reallocate Memory

- Best fit: more like "smallest fit", choose the smallest free'd memory.
- Worst fit: opposite choose criteria of best fit, choose the biggest.
- First fit: choose the first one that fit.
- Quick fit: group some part of free list together, for example (1st group for length less than 4 kB, 2nd group for length less than 8 kB). Faster than other, but hard to do coalescing. (You know when programmer slap the word "quick" in front of their algo, it means you will be fuk up real "quick".)

Lecture 10, Memory management 2

Paging

- Divied memory into parts. Each part call it a page.
- Make virtual address 2 parts, Virtual page number(VPN) and offset.
- MMU must translate from virtual page number to physical frame number(PFN).
- Simplest form is linear page table, which is only an array.
- Address translation is slow, since it has to do some reference (and access memory is slow).

Fast Translations(TLBs)

- Cache it. have 2 approaches hardware-based, software-based.
- Hardware-based: built-in instruction to handle TLB miss. Use page table base register(PTBR) to locate page table.
- Software-based: Let hardware raise an exception, OS does the rest.
- Some VPN can be the same, due to context switching.
- Some VPN may refer to the same PFN, e.g. shared libraries.
- TLB can fulled, use policies Least Recently Used(LRU) or Random to replace entry.

Page table

- Page table translates from VPN to PFN, and it has to remember a lot of page.
- It's too big or it has internal fragmentation.

- Multi-level page table. Got space trade with time (multi-level requires multiple memory referencing).

Lecture 11, Memory management 3

Page Swapping

- Since main memory can't hold infinite amount of pages, some of them must be store in the disk when not processing. Only page that is in the main memory can be processed. Accessing disk usually use long ass time, they usually want to avoid this.
- When processes try to find the page that is not in the main memory, Page fault exception is raised. OS has to handle that using page swapping.
- There are several page swapping algorithms such as First-in, first-out(FIFO), Clock, Least recently used(LRU).

Clean page

- Page that is in the main memory and not edited from the os. This page is identical to the one in the disk. Which means it is unnessary to write back to the disk.

Dirty page

- Opposite to the clean page, the page is edited and must write back to disk.

Zeroing

- When swap page, write zero to the memory page first then write the replacing page. For security (if not, the replaing page(process) will be able to read the old page).

Lecture 12, I/O Devices

I/O Devices

- Device that other than CPU, main memory. Usually relate to user more than other compute parts like mouse, keyboard and printer. Usually much slower than CPU many times.
- They send infomation to the computer, so CPU has to find somehow to get infomation using one of following method. Programmed I/O, Interrupt-driven I/O, I/O using DMA.

Programmed I/O

- The CPU continuously polls the device to see if it is ready, then the CPU issues a command.
- According to example in the slide. Probably use a thread to spin loop wait until that I/O device is ready.

Interrupt-driven I/O

- When I/O device is ready, raise hardware interrupt. The CPU will context switch to do what ever I/O device want.
- Context switch and interrupt handling create overhead.

Direct Memory Access(DMA)

- Let I/O devices access(read and write) main memory without using CPU. By using other hardware called DMA controller.
- There are several ways to communicate with I/O devices. Port-mapped I/O, Memory-mapped I/O, or both (Hybrid).

Port-mapped I/O

- Use 2 address spaces, 1 for main memory and another for I/O port.
- Use special instructions to write and read. Such as `IN REG, PORT` to read from the register to I/O port and `OUT PORT, REG` write from port to register shown in the slide. (Can't use `MOV` since they are in differnce address space)

Memory-mapped I/O

- Map memory for I/O port in main memory space.

Device drivers

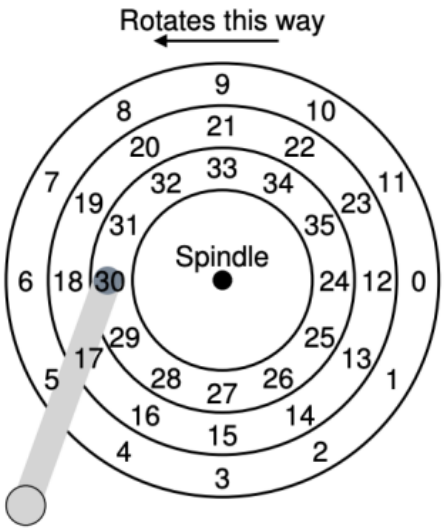
- Device-specific code for controlling the device. Often provided by hardware manufacturers.

Lecture 13, Disk Manegement

Disk

- Has many types such as Integrated Drive Electronics(IDE), Serial ATA(SATA).
- Has many way to represent its data such as Virture Geometry, Logical block addressing(LBA).
- Usually refer to hard disk drive that has disc(cylinder stuff) inside.
- Disk with cylinder inside, must have an arm to read a data on the disc.

Logical Representation



- Generally, the OS does not know the detail geometry of the drive.
- From now on, we will use this representation.

13

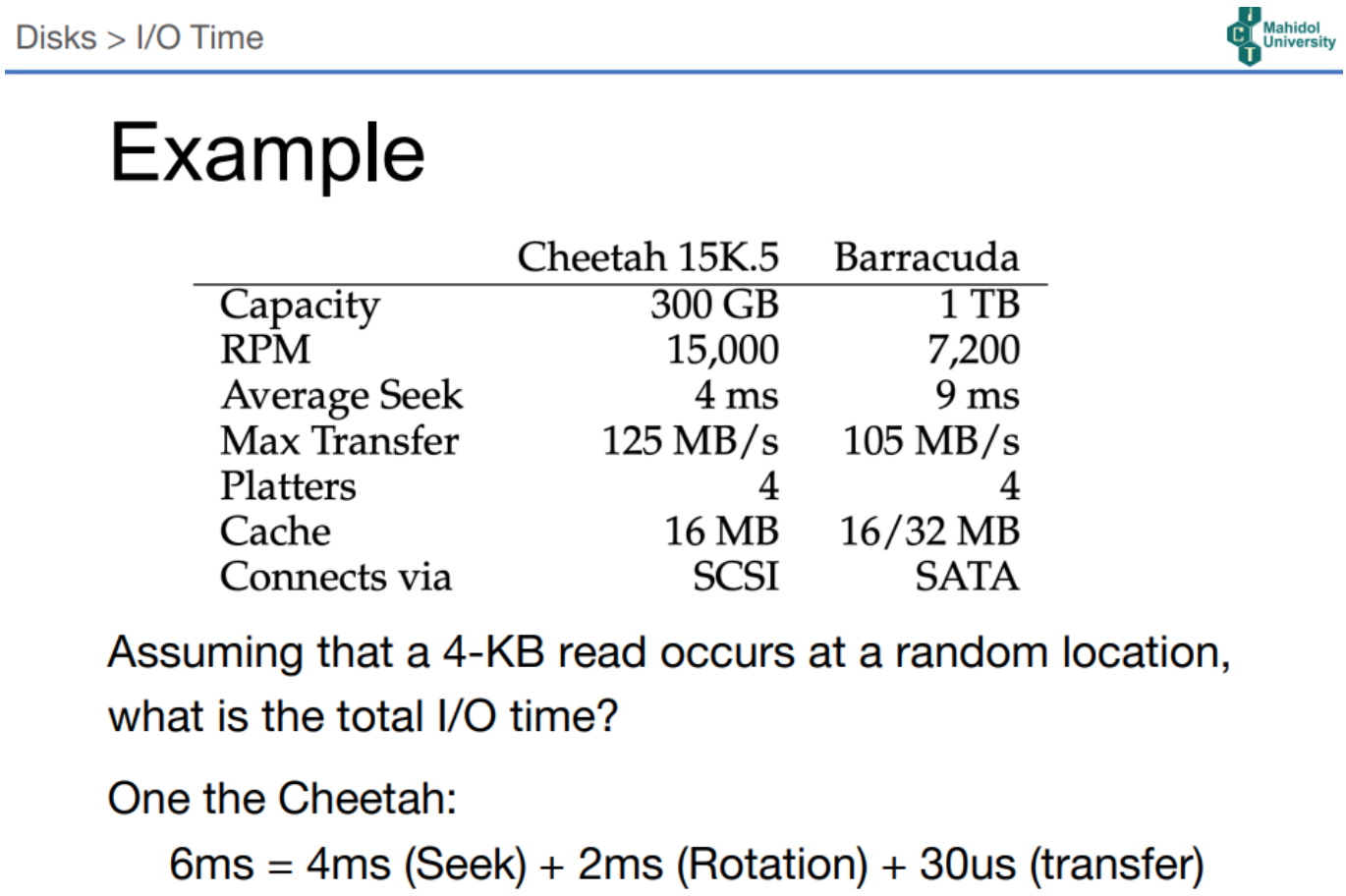
LBA representation of disk on the left from Lect. 13 page 13

Logical block addressing(LBA)

- Map each data point to a unique number(like an array).

Read/write time

- Rotational delay: a time for a desired to appear under the head. (Everything about rotation in physic works here.)
- Seek time: a time to move arms to a desired cylinder.
- Data transfer: a time for transferring data from/to disks.
- Seek time dominates the other two.
- Total time used for read/write = Rotational delay + Seek time + Data transfer.



Example of I/O time calculation (Use avg. rotation time := max rotation time/2) from Lect. 13 page 19

Disk Arm Scheduling Algorithms

- First Come First Served (FCFS): high arm movement
- Shortest Seek First (SSF): Low arm movement but has stavation problem.
- Elevator Algorithm: Up to the most top then down the most bottom. Lower arm movement than FCFS but higher than SSF and no stavation problem.
- Shortest Positioning Time First (SPTF): Take both head movement and rotation in to account to decited where it should go. Read the book

Lecture 14, Files and directories management

Files

- A file is simply a linear array of bytes, each of which we can read or write.
- Each file has a low-level name, often refer as `inode` number
- has many type
  - ASCII file
  - Binary file (has there own structure which specific for program that use them)
  - Directories has structure of the file system.
  - Characer special files (I/O serial devices).
  - Block special files (Disk devices).

Directories

- A type of file that contain list of files or other directories (pairs of readable name and low-level name).

File System(fs)

- File system is a method and data structure that the operating system uses to control how data is stored and retrieved.
- Example of file system, ext, ext2, ext3 ,ext4, FAT, minix, msdos, ncpfs nfs, ntfs, proc, Reiserfs and etc.

File System Interface

- Files
  - Creating files
  - Reading and writing files
  - Renaming files
  - Getting file information
  - Removing files
- Directories(Folders)
  - Making directories
  - Reading directories
  - Deleting directories

File Descriptor(fd)

- Integer that reference to the file.
- Private per process.

Creating Files

- open() system call API. It return file descriptor.
- Example `int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);`
- try `man open` on your linux terminal.

Reading and Writing Files

- `ssize_t read(int fd, void *buf, size_t count);` system call API for reading file. `read()` attempts to read up to `count` bytes from file descriptor `fd` into the buffer starting at `buf`. `read()` also move offset to offset + count. Return offset after read.
- `ssize_t write(int fd, const void *buf, size_t count);` system call API for reading file. write() writes up to `count` bytes from the buffer starting at `buf` to the file referred to by the file descriptor `fd`. `write()` also move offset to offset + count. Return offset after write.
- try `man read` and `man 2 write` on your linux terminal for more infomation.

Offset

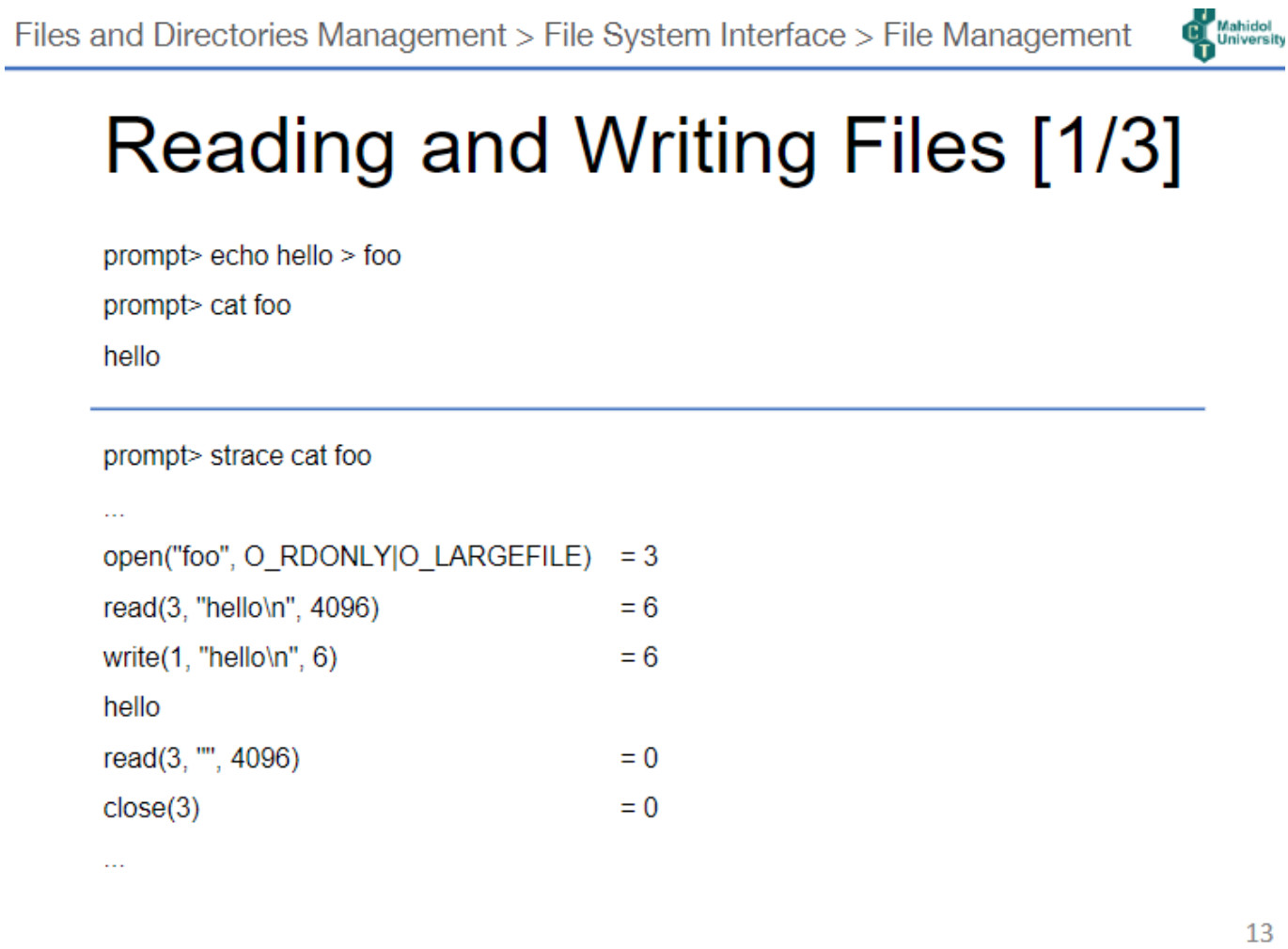
- When reading or writing file, there is a offset that point to at position that is reading or writing.
- `lseek()` system call API for setting read/write offset. Example usage: `lseek(fd, 200, SEEK_SET)` set offset of `fd` to be 200.
- See `man lseek` on your linux terminal for more infomation.

Force Write

- `write()` is not going to write at the moment it's called due to performance issue. It will buffer it for a while (or some conditions were met).
- use `fsync()` to force OS to write immediately.
- Example usage `fsync(fd);`
- See `man fsync` on your linux terminal for more infomation.

Trace system call APIs

- Use `strace $command` to see what happend when command is exacuted.



Example usage of `strace`

Renaming Files

- `rename(char *old, char *new)` system call API for rename file.
- Example usage `rename("foo.txt.tmp", "foo.txt")`
- `mv` unix terminal command for rename file.
- Example usage `mv foo.txt.tmp foo.txt`
- See `man 2 rename` and `man mv`. (`man rename` refer to `rename` linux command.)

Getting File Infomation

- Metadata of a file is infomation about that file. Such as size, Creation time, owner, hidden and so on.
- `stat()` or `fstat()` are system call API for see the metadata.
- `stat` is linux command to see the metadata of a file.

- See `man stat` for linux command and `man 2 stat` for system call API.

Removing File

- `unlink()` is system call API for remove a file.
- `rm` is linux command to remove a file.
- See `man 2 unlink` and `man rm`. (`man unlink` refer to `unlink` linux command.)

Making Directories

- When directory is created, it is empty.
- Empty directory has two entries, `.` (self) and `..` (parent directory).
- `mkdir()` is system call API for make a directory.
- `mkdir` is linux command to make a directory.
- See `man mkdir` for linux command and `man 2 mkdir` for system call API.

Reading Directories

- `opendir()` is system call API for open directory.
- `readdir()` is system call API for reading each file inside directory.
- See `man opendir` and `man readdir`.

Files and Directories Management > File System Interface > Directory Management

Reading Directories

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%lu %s\n", (unsigned long) d->d_ino, d->d_name);
    }
    closedir(dp);
    return 0;
}

struct dirent {
    char           d_name[256]; /* filename */
    ino_t          d_ino;       /* inode number */
    off_t          d_off;       /* offset to the next dirent */
    unsigned short d_reclen;    /* length of this record */
    unsigned char  d_type;      /* type of file */
};
```

26

Example usage of `opendir` and `readdir`

Deleting Directories

- Deleting a directory require that directory to be empty. (Unless you use -r recursive delete.)
- `rmdir()` is system call API for delete a directory.
- `rmdir` is linux command to delete a directory.
- See `man rmdir` for linux command and `man 2 rmdir` is system call API.

File Links

- Hard Links
- Soft Links

Hard Links

- Hard link creates a new file that has the same reference as the original. (has same inode)
- When create a hard link, the reference count goes up by one. When `unlink()` is called and reference count reaches zero, the file system free the inode and related data blocks and thus truly "delete" the file.
- Can't create one for directory.
- Can't use hard link across disk, since inode number is specific for each file system.
- `link()` system call API for create hard link. Example usage `link("file", "file2");`, `file2` is created which has the same inode as `file`.
- `ln` is linux command to create hard link. Example usage `ln file file2`, `file2` is created which has the same inode as `file`.
- See `man 2 link` and `man ln`.

Soft Links(Symbolic Links)

- Soft links creates a new file that refer to the original file (contain its path). Like shortcuts in windows.
- If the original file is moved, the reference won't move along (dangling reference).
- `ln -s` is linux command for create a soft link. Example usage `ln -s file file2`, create `file2` as soft link to `file`.
- See `man ln`.

File Information

- A file has metadata about itself. Like file type, permission, link count, owner, group, size, modification datetime, file name and etc.
- `ls -l` or `stat` to look into it.

File Permission

- Permission for user(the one who create the file), group and other(anyone else from the group) (u, g, o).
- Permission to read, write and execute (r, w, x).
- File permission, which usually packed with file type, often writen out in this order `drwxrwxrwx`. The first `d` is means that file is directory. The first `rwx` from the left is read, write and execute allowance for user. The second `rwx` from the left is read, write and execute allowance for group. The third `rwx` from the left is read, write and execute allowance for other.
- `-` is use for absent of that permission.
- Most of the time, the file has only some permission. Such as it is a file that let user and group execute only, can't read or write. It would have permission like this `---x--x---`.

Setting Permission Bits

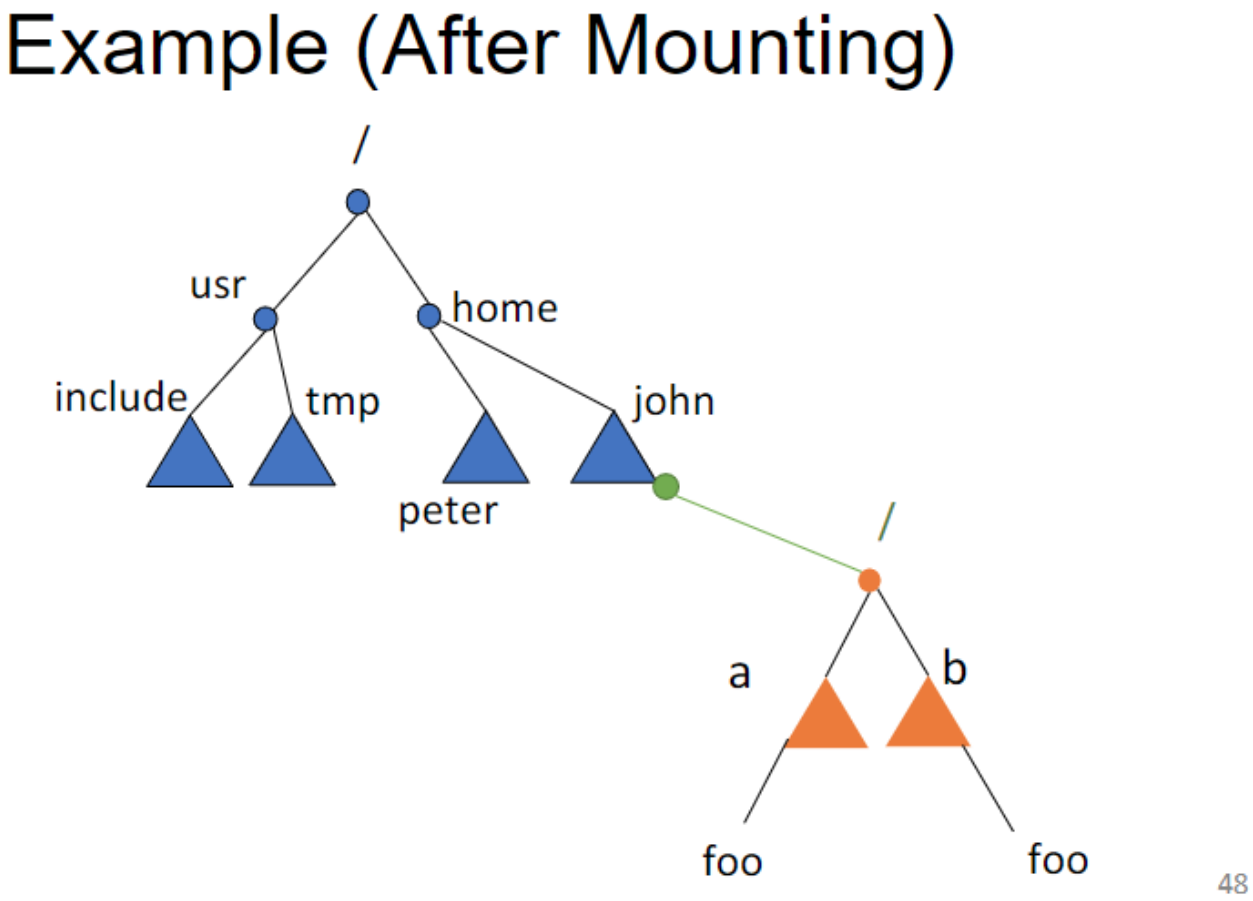
- `chmod` is system call for change file permission.
- Symbolic mode
  - `chmod g+x foo.txt` means allow(+) execute(x) to group(g) of `foo.txt`.
  - `chmod ug-r foo.txt` means disallow(-) read(r) to user(u) and group(g) of `foo.txt`.
  - `chmod +w foo.txt` means allow(+) write(w) to all of `foo.txt`.
- Absolute mode
  - We can represent a set of permission(rwx) as three digits binary. Mapping r-- -> 100, -w- -> 010 and --x -> 001 (anything else just add it together, for ex. rw- -> 110 and so on).
  - Then we can use convert that number to be octal(base-8), we get number 0 to 7 that can represent every permutation of permission. For a type of user.
  - Use 3 octal numbers to represent all 3 user, group and other respectively. For ex. 700 -> 111000000 -> `rwx------` (ignore `d` for now, it is directory or not is already detemine when we create that file).
  - `chmod 700 foo.txt` means set permission of `foo.txt` to be `rwx------`.
  - `chmod 644 foo.txt` means set permission of `foo.txt` to be `rw-r--r--`.

Access Control List(ACL)

- ACL represent exactly who can access a given resource.

Making and Mounting File System

- Making means to format devices to use file system specified (Write an empty file system). For example, `mkfs -t ext3 /dev/sda1` means format `/dev/sda1` to use `ext3`. `/dev/sda1` will be empty after `mkfs` is complete.
- Mounting means to make other devices accessable via already existing file system. Like, external hard drives. For example, `mount -t ext3 /dev/sda1 /home/john` means mount `/dev/sda1` (that use `ext3` file system) to `/home/john`. On success, content inside of `/dev/sda1` can be access via `/home/john`.
- See `man mkfs` and `man mount`.



The green edge is created when `mount` was called; Orange fs can be access via path `/home/john`

Lecture 15, File System Implementation

File System

- Main aspects
  - Data structure, how file was stored.
  - Access method, how open(), read(), write() is implemented.
- In this note, we will talk about very simple file system: vsfs.

File System Organization

- Can be vistualize as an array. Seperate into blocks(sections).
- User's data store at Data region(D).
- Metadata is store at inode table(I), used for track information about each file.
  - Each inode is around 128 or 256 bytes.
  - Maximum of inode in the system also refer to maximum number of file in the system (kind of, since we can have file with the same inode).
- Data bitmap(d) and inode bitmap(i) are used to track whether inodes or data blocks are free or allocated.
- Superblock(S) metadata of system such as where the inode table begin, inodes count, data block count and etc.

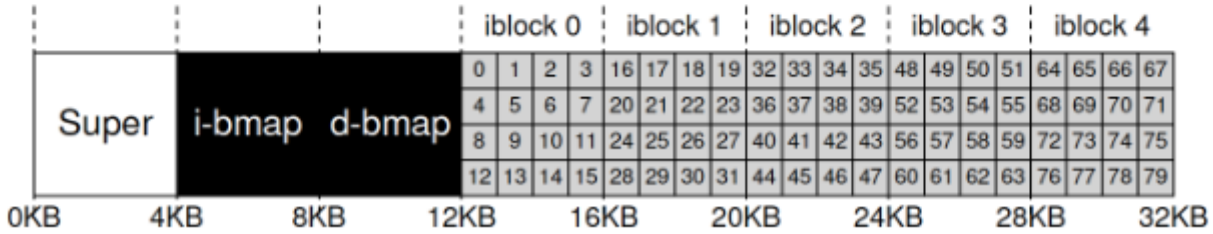
Index Node(inode, i-number) (finally, they explain this shit)

- Each inode is a number, which is low-level name of the file.
- In vsfs, given an i-number, we can directly calculate where on the disk the corresponding inode is located.

File System Implementation > File Organization

Example

- To read inode # 32,
  - calculate the offset into the inode region ⍰ 32 * *sizeof(inode)* = 32 * 256 = 8192
  - 12 KB + 8192 = 20 KB
  - Disks are not byte addressable, but rather consist of a large number of addressable sectors, usually 512 bytes
  - To fetch the block of inodes that contains inode 32 = (20 * 1024) / 512 = sector # 40



18

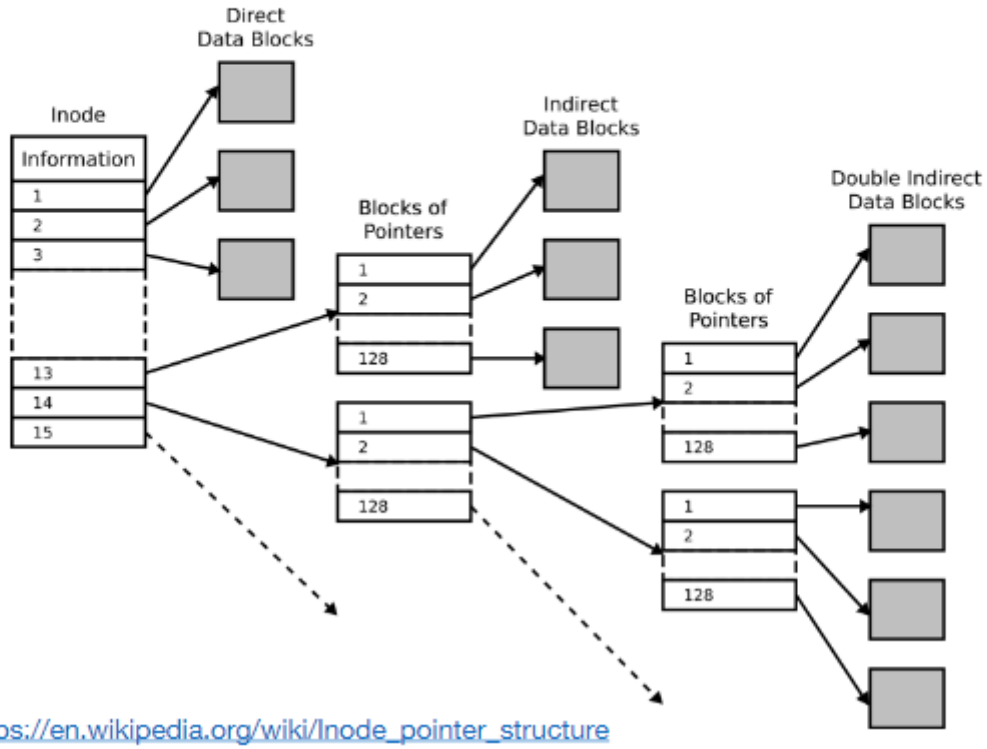
Example of calculate inode position.

Multi-Level Index

- Direct Pointer
  - inode that point to a file.
  - It is limited to file that smaller to a block.
- Indirect Pointer
  - inode that point to a block that store pointers that point to the file.
  - it may store several pointer. Which can use it to store file that bigger than a block. It still has limitation though just bigger size than Direct pointer.
  - We can just use pointer to pointer to...to file if we don't have enough.

File System Implementation > File Organization > Multi-Level Index

Multi-Level Index



Source: https://en.wikipedia.org/wiki/Inode_pointer_structure

24

inode pointer structure.

Directory Organization

- File systems treat directories as a special type of file.
- A directory contains a list of pairs (entry name, inode). Somewhere in the inode table (with the type field of the inode marked as "directory" instead of "regular file").

Free Space Management

- A file system must reference count of each inode. If reference count reachs zero, mark that space as free space or some sort.
- In vsfs, they use data and inode bitmaps to keep track.

Access Path

- Each system call has to read or write into its inode or bitmap(related to create or delete file) in someway.
- For example To create a file:
    - one read to the inode bitmap. (to check if it already exist)
    - one write to the inode bitmap. (write that new one will be alocate)
    - one write to the new inode itself (to initialize it).
    - one to the data of the directory.
    - one read and write to the directory inode to update it.

Writing A File To Disk [1/2]

- To create a file:
  - one read to the inode bitmap.
  - one write to the inode bitmap.
  - one write to the new inode itself (to initialize it).
  - one to the data of the directory.
  - one read and write to the directory inode to update it.
- Each write to a file logically generates five I/Os
  - one to read the data bitmap
  - one to write the bitmap
  - two more to read and write the inode
  - one to write the actual block itself.

32

Writing A File To Disk [2/2]

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
create (/foo/bar)		read write	read	read write	read write	read	read	write		
write()	read write				read write			write		
write()	read write				read write				write	
write()	read write				read write					write

33

Access path examples.

Caching (File System)

- Modern systems employ a dynamic partitioning(IDK what does this mean as well, lmao) approach to integrate virtual pages and file system pages into a unified page cache.

Buffering (File System)

- Write is slow process. By delaying writes, the file system can group multiple of write call together, and make it more efficient.

Lecture 16, File System Optimization

Bad methods

- Threated disk as Random-access memory.
- Too small block size (512 Bytes).
- Poor free space allocation.

Fast File System(FFS)

- Divied disk to be groups. Each group whole stucture Superblock, inode bitmap, data bitmap, inode table and data region.
- Key idea is to keep related stuff together (to reduce revolution time).
- Directories: find a (cylinder) group with a low number of allocated directory and high number of free inodes.
- Files: Allocate inode and data in the same group. Place files that in the same group as its directory they are in.

Cite

- OSTEP Book
- ITCS343 Principles Of Operating Systems' lectures slide.
- Wikipedia.
- Linux Programmer's Manual `man`.